

GROUP EXERCISE — SOLUTIONS

ECE422 Computer Communications & Networking · Shannon · Checksum · CRC · Parity · Coding

Fields in Q2 are hexadecimal: a2 : 06 : 23 → 10100010 : 00000110 : 00100011

1. A radio link at 40 KHz can transmit a signal at 100 mW and the channel has 10 mW of noise.

What is the capacity of the channel?

$$S/N = 100 \text{ mW} / 10 \text{ mW} = \mathbf{10} \quad (= 10 \cdot \log_{10} 10 = 10 \text{ dB})$$

$$C = B \cdot \log_2(1 + S/N) = 40,000 \cdot \log_2(11)$$

$$\log_2(11) = \log_{10}(11) / \log_{10}(2) = 1.0414 / 0.3010 \approx 3.46$$

$$C \approx 40,000 \times 3.46 \approx \mathbf{138.4 \text{ kbps}}$$

2. Show the 8-bit checksum that will be transmitted to protect the fields a2:06:23.
-

<pre>a2 = 1010 0010 (162) + 06 = 0000 0110 (6) + 23 = 0010 0011 (35) ----- cb = 1100 1011 (203) no carry out of 8 bits → nothing to wrap Complement: 0011 0100 = 0x34 ← CHECKSUM Receiver: cb + 34 = 1111 1111 → complement = 0000 0000 → OK</pre>

Checksum = 0011 0100 (0x34). Transmitted: a2 : 06 : 23 : 34

3. Show 3 possible field values that would generate the same checksum as in 2).
-

The checksum depends only on the ones-complement **sum** (= cb). Any fields adding to cb give checksum 34:

#	Fields	Why
1	23 : 06 : a2	same values, different order (addition is commutative)
2	a3 : 05 : 23	+1 in one field, -1 in another
3	cb : 00 : 00	cb + 0 + 0 = cb

Lesson: a checksum cannot detect reordered fields or errors that cancel out.

4. What is the CRC field content in Q2 if $C(x) = 1101$?

$C(x) = 1101 = x^3 + x^2 + 1 \rightarrow$ degree $r = 3 \rightarrow$ append **000**.

Dividend: 10100010 00000110 00100011 **000**. Only XOR steps shown: whenever the leading bit is 1, XOR 1101 under it (windows starting with 0 are skipped).

bit 1:	1010	⊗	1101	=	0111
bit 2:	1110	⊗	1101	=	0011
bit 4:	1101	⊗	1101	=	0000
bit 14:	1100	⊗	1101	=	0001
bit 17:	1010	⊗	1101	=	0111
bit 18:	1110	⊗	1101	=	0011
bit 20:	1101	⊗	1101	=	0000
bit 24:	1000	⊗	1101	=	0101
remainder (last 3 bits) = 101					

CRC field = 101 $\rightarrow$ transmitted: 101000100000011000100011 **101** (dividing it by 1101 leaves 000 ✓)

5. If we use parity check for a 5-bit word, how many 2-bit errors may be undetectable?

A single parity bit misses **every** even number of bit errors $\rightarrow$ all 2-bit errors are undetectable. Count = number of ways to pick 2 bits:

- 5-bit word including the parity bit (like 1001P): $C(5,2) = 5 \cdot 4 / 2 = 10$
- 5 data bits + 1 parity bit (6 bits): $C(6,2) = 6 \cdot 5 / 2 = 15$ (*state your assumption*)

6. If we use 1 bit for error detection of 5-bit words, which of 1-bit parity (assuming up to 2-bit errors) or CRC would have fewer non-detectable errors?

Step 1 – Which CRC? 1 check bit → $C(x)$ has degree 1 → 2-bit generator: **10** (x) or **11** ($x + 1$). 10 always leaves remainder 0 (useless), so the 1-bit CRC must use **$C(x) = 11$** .

Step 2 – Compute both check bits for data 10110.

Parity (even): 1 0 1 1 0 → three 1s (odd) → parity bit = 1

CRC with 11: append 1 zero → 101100

```

101100
11      XOR → 011100
11      XOR → 000100
11      XOR → 000010
remainder = 1      → CRC bit = 1    (same as parity!)

```

Sent frame (both schemes): 10110 1

Step 3 – Receiver side, same frame 101101.

Case	Received	Parity check (# of 1s)	CRC check ($\div 11$ remainder)	Result
No error	101101	4 → even ✓	0 ✓	OK
1-bit error (bit 1)	001101	3 → odd ✗	1 ✗	both detect
2-bit error (bits 1,2)	011101	4 → even ✓	0 ✓	both MISS

Why always the same: each XOR with 11 flips two neighbouring bits → the number of 1s changes by 0 or 2 → odd stays odd, even stays even. The final 1-bit remainder is therefore 1 exactly when the number of 1s is odd = the even-parity bit.

Step 4 – Count (6 bits = 5 data + 1 check).

	1-bit parity	1-bit CRC (11)
1-bit errors: $C(6,1) = 6$	all 6 detected	all 6 detected
2-bit errors: $C(6,2) = 6 \cdot 5 / 2 = 15$	15 undetected	15 undetected

Answer: neither — both have 15 non-detectable errors. With only 1 bit, CRC becomes parity; CRC is stronger only with more check bits.

7. How many 2-bit errors can be undetected by checksum in 5-bit words?

Assumption: one 5-bit data word + its 5-bit checksum are sent (10 bits).

Step 1 – Example.

data	D = 10110	
checksum	C = 01001	(complement of D)
D + C	= 11111	→ complement = 00000 → no error

Step 2 – Why some errors are missed. C is the complement of D, so in each bit position one of them is 0 and the other is 1. If the same position is flipped in both, the 0 becomes 1 and the 1 becomes 0 → D + C is still 11111 → the receiver sees no error.

Step 3 – All undetected 2-bit errors.

Position flipped in D and C	D received	C received	D + C
1	00110	11001	11111
2	11110	00001	11111
3	10010	01101	11111
4	10100	01011	11111
5	10111	01000	11111

Every other 2-bit error changes the sum and is detected (e.g. flipping bits 1 and 2 of D: 01110 + 01001 = 10111 ≠ 11111).

Answer: 5 undetectable 2-bit errors (one per bit position) out of $C(10,2) = 45$ possible 2-bit errors.

8. What is the CRC field content of the string in Q2 if $C(x) = 1000101$?

$C(x) = 1000101 = x^6 + x^2 + 1 \rightarrow$ degree $r = 6 \rightarrow$ append **000000**. Only XOR steps shown:

bit 1:	1010001	⊗	1000101	=	0010100
bit 3:	1010000	⊗	1000101	=	0010101
bit 5:	1010100	⊗	1000101	=	0010001
bit 7:	1000100	⊗	1000101	=	0000001
bit 13:	1110001	⊗	1000101	=	0110100
bit 14:	1101000	⊗	1000101	=	0101101
bit 15:	1011010	⊗	1000101	=	0011111
bit 17:	1111101	⊗	1000101	=	0111000
bit 18:	1110001	⊗	1000101	=	0110100
bit 19:	1101000	⊗	1000101	=	0101101
bit 20:	1011010	⊗	1000101	=	0011111
bit 22:	1111100	⊗	1000101	=	0111001
bit 23:	1110010	⊗	1000101	=	0110111
bit 24:	1101110	⊗	1000101	=	0101011
remainder (last 6 bits) = 101011					

CRC field = 101011 → transmitted: 101000100000011000100011 **101011** (remainder check = 000000

✓)

9. (Optional) Design a coding scheme to transmit 4-bit words (16 combinations) where a string of words cannot contain more than 4 consecutive 1s or 0s.

Idea (like 4B/5B): map each 4-bit word to a **5-bit code** that does **not start** and does **not end** with three equal bits.

Then at the boundary between two codes at most $2 + 2 = 4$ equal bits can line up, and inside a code the longest run is 3.

18 of the 32 five-bit codes satisfy the rule ($32 - 8 - 8 + 2$); choose 16:

Data	Code	Data	Code	Data	Code	Data	Code
0000	00100	0100	01010	1000	01110	1100	10100
0001	00101	0101	01011	1001	10001	1101	10101
0010	00110	0110	01100	1010	10010	1110	10110
0011	01001	0111	01101	1011	10011	1111	11001

Checked: every pair of codes back-to-back has a longest run of 4. Efficiency = $4/5 = 80\%$. The 2 unused codes (11010, 11011) can serve as control codes.

Key rules: dB $\rightarrow$ S/N: divide by 10, take 10 to that power · checksum: add, wrap carry, complement · CRC: append r zeros ($r = \text{degree} = \text{bits of } C(x) - 1$), XOR-divide, remainder = CRC · parity misses all even-bit errors, count with $C(n,k)$.